5
3D Math for Games

A game is a mathematical model of a virtual world simulated in real time on a computer of some kind. Therefore, mathematics pervades everything we do in the game industry. Game programmers make use of virtually all branches of mathematics, from trigonometry to algebra to statistics to calculus. However, by far the most prevalent kind of mathematics you'll be doing as a game programmer is 3D vector and matrix math (i.e., 3D *linear algebra*).

Even this one branch of mathematics is very broad and very deep, so we cannot hope to cover it in any great depth in a single chapter. Instead, I will attempt to provide an overview of the mathematical tools needed by a typical game programmer. Along the way, I'll offer some tips and tricks, which should help you keep all of the rather confusing concepts and rules straight in your head. For an excellent in-depth coverage of 3D math for games, I highly recommend Eric Lengyel's book [32] on the topic. Chapter 3 of Christer Ericson's book [14] on real-time collision detection is also an excellent resource.

5.1 Solving 3D Problems in 2D

Many of the mathematical operations we're going to learn about in the following chapter work equally well in 2D and 3D. This is very good news, because

it means you can *sometimes* solve a 3D vector problem by thinking and drawing pictures in 2D (which is considerably easier to do!) Sadly, this equivalence between 2D and 3D does not hold all the time. Some operations, like the cross product, are only defined in 3D, and some problems only make sense when all three dimensions are considered. Nonetheless, it almost never hurts to start by thinking about a simplified two-dimensional version of the problem at hand. Once you understand the solution in 2D, you can think about how the problem extends into three dimensions. In some cases, you'll happily discover that your 2D result works in 3D as well. In others, you'll be able to find a coordinate system in which the problem really *is* two-dimensional. In this book, we'll employ two-dimensional diagrams wherever the distinction between 2D and 3D is not relevant.

5.2 Points and Vectors

The majority of modern 3D games are made up of three-dimensional objects in a virtual world. A game engine needs to keep track of the positions, orientations and scales of all these objects, animate them in the game world, and transform them into screen space so they can be rendered on screen. In games, 3D objects are almost always made up of triangles, the vertices of which are represented by points. So, before we learn how to represent whole objects in a game engine, let's first take a look at the point and its closely related cousin, the vector.

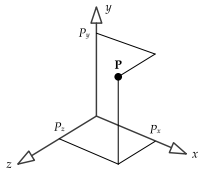

Figure 5.1. A point represented in Cartesian coordinates.

5.2.1 Points and Cartesian Coordinates

Technically speaking, a *point* is a location in n-dimensional space. (In games, n is usually equal to 2 or 3.) The Cartesian coordinate system is by far the most common coordinate system employed by game programmers. It uses two or three mutually perpendicular axes to specify a position in 2D or 3D space. So, a point **P** is represented by a pair or triple of real numbers, (P_x, P_y) or (P_x, P_y, P_z) (see Figure 5.1).

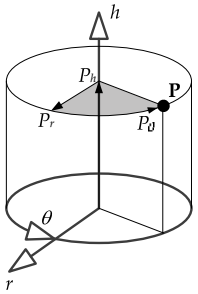

Figure 5.2. A point represented in cylindrical coordinates.

Of course, the Cartesian coordinate system is not our only choice. Some other common systems include:

- *Cylindrical coordinates*. This system employs a vertical "height" axis h, a radial axis r emanating out from the vertical, and a yaw angle *theta* (θ). In cylindrical coordinates, a point **P** is represented by the triple of numbers (P_h, P_r, P_θ). This is illustrated in Figure 5.2.

- *Spherical coordinates*. This system employs a pitch angle phi (ϕ), a yaw angle theta (θ) and a radial measurement r. Points are therefore represented by the triple of numbers (P_r, P_ϕ, P_θ). This is illustrated in Figure 5.3.

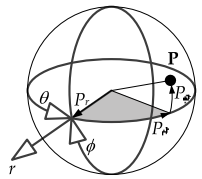

Figure 5.3. A point represented in spherical coordinates.

Cartesian coordinates are by far the most widely used coordinate system in game programming. However, always remember to select the coordinate system that best maps to the problem at hand. For example, in the game *Crank the Weasel* by Midway Home Entertainment, the main character Crank runs around an art-deco city picking up loot. I wanted to make the items of loot swirl around Crank's body in a spiral, getting closer and closer to him until they disappeared. I represented the position of the loot in cylindrical coordinates relative to the Crank character's current position. To implement the spiral animation, I simply gave the loot a constant angular speed in θ, a small constant linear speed inward along its radial axis r and a very slight constant linear speed upward along the h-axis so the loot would gradually rise up to the level of Crank's pants pockets. This extremely simple animation looked great, and it was much easier to model using cylindrical coordinates than it would have been using a Cartesian system.

5.2.2 Left-Handed versus Right-Handed Coordinate Systems

In three-dimensional Cartesian coordinates, we have two choices when arranging our three mutually perpendicular axes: right-handed (RH) and left-handed (LH). In a right-handed coordinate system, when you curl the fingers of your right hand around the z-axis with the thumb pointing toward positive z coordinates, your fingers point from the x-axis toward the y-axis. In a left-handed coordinate system the same thing is true using your left hand.

The only difference between a left-handed coordinate system and a right-

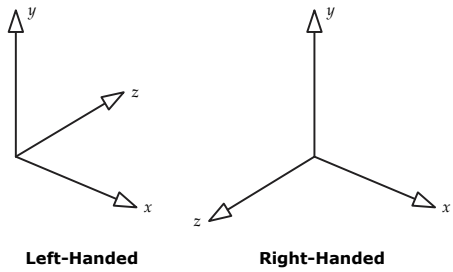

Figure 5.4. Left- and right-handed Cartesian coordinate systems.

handed coordinate system is the direction in which one of the three axes is pointing. For example, if the y-axis points upward and x points to the right, then z comes toward us (out of the page) in a right-handed system, and away from us (into the page) in a left-handed system. Left- and right-handed Cartesian coordinate systems are depicted in Figure 5.4.

It is easy to convert from left-handed to right-handed coordinates and vice versa. We simply flip the direction of any one axis, leaving the other two axes alone. It's important to remember that the rules of mathematics do not change between left-handed and right-handed coordinate systems. Only our *interpretation* of the numbers—our mental image of how the numbers map into 3D space—changes. Left-handed and right-handed conventions apply to visualization only, not to the underlying mathematics. (Actually, handedness does matter when dealing with cross products in physical simulations, because a cross product is not actually a vector—it's a special mathematical object known as a *pseudovector*. We'll discuss pseudovectors in a little more depth in Section 5.2.4.9.)

The mapping between the numerical representation and the visual representation is entirely up to us as mathematicians and programmers. We could choose to have the y-axis pointing up, with z forward and x to the left (RH) or right (LH). Or we could choose to have the z-axis point up. Or the x-axis could point up instead—or down. All that matters is that we decide upon a mapping, and then stick with it consistently.

That being said, some conventions do tend to work better than others for certain applications. For example, 3D graphics programmers typically work with a left-handed coordinate system, with the y-axis pointing up, x to the right and positive z pointing away from the viewer (i.e., in the direction the virtual camera is pointing). When 3D graphics are rendered onto a 2D screen using this particular coordinate system, increasing z-coordinates correspond to increasing *depth* into the scene (i.e., increasing distance away from the virtual camera). As we will see in subsequent chapters, this is exactly what is required when using a z-buffering scheme for depth occlusion.

5.2.3 Vectors

A *vector* is a quantity that has both a *magnitude* and a *direction* in n-dimensional space. A vector can be visualized as a *directed line segment* extending from a point called the *tail* to a point called the *head*. Contrast this to a *scalar* (i.e., an ordinary real-valued number), which represents a magnitude but has no direction. Usually scalars are written in italics (e.g., v) while vectors are written in boldface (e.g., $\mathbf{v}$).

A 3D vector can be represented by a triple of scalars (x, y, z), just as a point

can be. The distinction between points and vectors is actually quite subtle. Technically, a vector is just an offset *relative to* some known point. A vector can be moved anywhere in 3D space—as long as its magnitude and direction don't change, it is the same vector.

A vector can be used to represent a point, provided that we fix the tail of the vector to the origin of our coordinate system. Such a vector is sometimes called a *position vector* or *radius vector*. For our purposes, we can interpret any triple of scalars as either a point or a vector, provided that we remember that a *position vector* is constrained such that its tail remains at the origin of the chosen coordinate system. This implies that points and vectors are treated in subtly different ways mathematically. One might say that points are *absolute*, while vectors are *relative*.

The vast majority of game programmers use the term "vector" to refer both to points (position vectors) and to vectors in the strict linear algebra sense (purely directional vectors). Most 3D math libraries also use the term "vector" in this way. In this book, we'll use the term "direction vector" or just "direction" when the distinction is important. Be careful to always keep the difference between points and directions clear in your mind (even if your math library doesn't). As we'll see in Section 5.3.6.1, directions need to be treated differently from points when converting them into homogeneous coordinates for manipulation with 4×4 matrices, so getting the two types of vector mixed up can and will lead to bugs in your code.

5.2.3.1 Cartesian Basis Vectors

It is often useful to define three *orthogonal unit vectors* (i.e., vectors that are mutually perpendicular and each with a length equal to one), corresponding to the three principal Cartesian axes. The unit vector along the x-axis is typically called $\mathbf{i}$, the y-axis unit vector is called $\mathbf{j}$, and the z-axis unit vector is called $\mathbf{k}$. The vectors $\mathbf{i}$, $\mathbf{j}$ and $\mathbf{k}$ are sometimes called Cartesian *basis vectors*.

Any point or vector can be expressed as a sum of scalars (real numbers) multiplied by these unit basis vectors. For example,

$$(5, 3, -2) = 5\mathbf{i} + 3\mathbf{j} - 2\mathbf{k}.$$

5.2.4 Vector Operations

Most of the mathematical operations that you can perform on scalars can be applied to vectors as well. There are also some new operations that apply only to vectors.

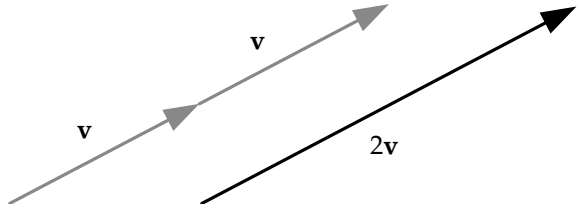

Figure 5.5. Multiplication of a vector by the scalar 2.

5.2.4.1 Multiplication by a Scalar

Multiplication of a vector **a** by a scalar s is accomplished by multiplying the individual components of **a** by s:

$$s\mathbf{a} = (sa_x, sa_y, sa_z).$$

Multiplication by a scalar has the effect of scaling the magnitude of the vector, while leaving its direction unchanged, as shown in Figure 5.5. Multiplication by -1 flips the direction of the vector (the head becomes the tail and vice versa).

The scale factor can be different along each axis. We call this *nonuniform scale*, and it can be represented as the *component-wise product* of a scaling vector **s** and the vector in question, which we'll denote with the $\otimes$ operator. Technically speaking, this special kind of product between two vectors is known as the *Hadamard product*. It is rarely used in the game industry—in fact, nonuniform scaling is one of its *only* commonplace uses in games:

$$\mathbf{s} \otimes \mathbf{a} = (s_x a_x, s_y a_y, s_z a_z). \tag{5.1}$$

As we'll see in Section 5.3.7.3, a scaling vector **s** is really just a compact way to represent a 3×3 diagonal scaling matrix **S**. So another way to write Equation (5.1) is as follows:

$$\mathbf{aS} = \begin{bmatrix} a_x & a_y & a_z \end{bmatrix} \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & s_z \end{bmatrix} = \begin{bmatrix} s_x a_x & s_y a_y & s_z a_z \end{bmatrix}.$$

We'll explore matrices in more depth in Section 5.3.

5.2.4.2 Addition and Subtraction

The addition of two vectors **a** and **b** is defined as the vector whose components are the sums of the *components* of **a** and **b**. This can be visualized by placing

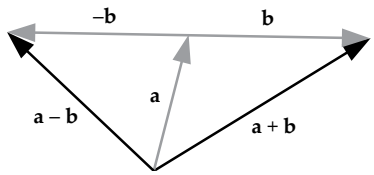

Figure 5.6. Vector addition and subtraction.

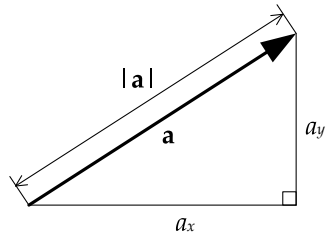

Figure 5.7. Magnitude of a vector (shown in 2D for ease of illustration).

the head of vector **a** onto the tail of vector **b**—the sum is then the vector from the tail of **a** to the head of **b** (see also Figure 5.6):

$$\mathbf{a} + \mathbf{b} = \left[(a_x + b_x), (a_y + b_y), (a_z + b_z)\right].$$

Vector subtraction $\mathbf{a} - \mathbf{b}$ is nothing more than addition of **a** and $-\mathbf{b}$ (i.e., the result of scaling **b** by -1, which flips it around). This corresponds to the vector whose components are the difference between the components of **a** and the components of **b**:

$$\mathbf{a} - \mathbf{b} = \left[(a_x - b_x), (a_y - b_y), (a_z - b_z)\right].$$

Vector addition and subtraction are depicted in Figure 5.6.

Adding and Subtracting Points and Directions

You can add and subtract direction vectors freely. However, technically speaking, points cannot be added to one another—you can only add a direction vector to a point, the result of which is another point. Likewise, you can take the difference between two points, resulting in a direction vector. These operations are summarized below:

- direction + direction = direction
- direction – direction = direction
- point + direction = point
- point – point = direction
- point + point = *nonsense*

5.2.4.3 Magnitude

The magnitude of a vector is a scalar representing the length of the vector as it would be measured in 2D or 3D space. It is denoted by placing vertical bars

around the vector's boldface symbol. We can use the Pythagorean theorem to calculate a vector's magnitude, as shown in Figure 5.7:

$$|\mathbf{a}| = \sqrt{a_x^2 + a_y^2 + a_z^2}.$$

5.2.4.4 Vector Operations in Action

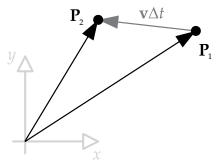

Figure 5.8. Simple vector addition can be used to find a character's position in the next frame, given her position and velocity in the current frame.

Believe it or not, we can already solve all sorts of real-world game problems given just the vector operations we've learned thus far. When trying to solve a problem, we can use operations like addition, subtraction, scaling and magnitude to generate new data out of the things we already know. For example, if we have the current position vector of an AI character $\mathbf{P}_1$, and a vector $\mathbf{v}$ representing her current velocity, we can find her position on the next frame $\mathbf{P}_2$ by scaling the velocity vector by the frame time interval Δt, and then adding it to the current position. As shown in Figure 5.8, the resulting vector equation is $\mathbf{P}_2 = \mathbf{P}_1 + \mathbf{v}\,\Delta t$. (This is known as *explicit Euler integration*—it's actually only valid when the velocity is constant, but you get the idea.)

As another example, let's say we have two spheres, and we want to know whether they intersect. Given that we know the center points of the two spheres, $\mathbf{C}_1$ and $\mathbf{C}_2$, we can find a direction vector between them by simply subtracting the points, $\mathbf{d} = \mathbf{C}_2 - \mathbf{C}_1$. The magnitude of this vector $d = |\mathbf{d}|$ determines how far apart the spheres' centers are. If this distance is less than the sum of the spheres' radii, they are intersecting; otherwise they're not. This is shown in Figure 5.9.

Square roots are expensive to calculate on most computers, so game programmers should always use the *squared magnitude* whenever it is valid to do

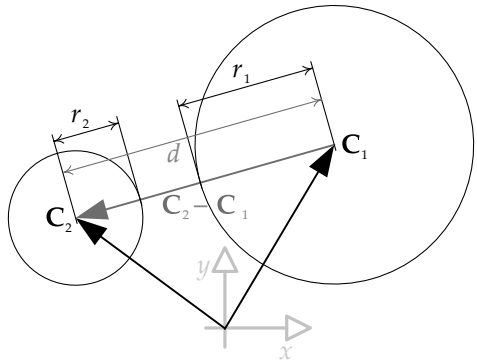

Figure 5.9. A sphere-sphere intersection test involves only vector subtraction, vector magnitude and floating-point comparison operations.

so:

$$|\mathbf{a}|^2 = (a_x^2 + a_y^2 + a_z^2).$$

Using the squared magnitude is valid when comparing the *relative lengths* of two vectors ("is vector **a** longer than vector **b**?"), or when comparing a vector's magnitude to some other (squared) scalar quantity. So in our sphere-sphere intersection test, we should calculate $d^2 = |\mathbf{d}|^2$ and compare this to the squared sum of the radii, $(r_1 + r_2)^2$ for maximum speed. When writing high-performance software, never take a square root when you don't have to!

5.2.4.5 Normalization and Unit Vectors

A *unit vector* is a vector with a magnitude (length) of one. Unit vectors are very useful in 3D mathematics and game programming, for reasons we'll see below.

Given an arbitrary vector **v** of length $v = |\mathbf{v}|$, we can convert it to a unit vector **u** that points in the same direction as **v**, but has unit length. To do this, we simply multiply **v** by the reciprocal of its magnitude. We call this *normalization:*

$$\mathbf{u} = \frac{\mathbf{v}}{|\mathbf{v}|} = \frac{1}{v}\mathbf{v}.$$

5.2.4.6 Normal Vectors

A vector is said to be *normal* to a surface if it is *perpendicular* to that surface. Normal vectors are highly useful in games and computer graphics. For example, a *plane* can be defined by a point and a normal vector. And in 3D graphics, lighting calculations make heavy use of normal vectors to define the direction of surfaces relative to the direction of the light rays impinging upon them.

Normal vectors are usually of unit length, but they do not need to be. Be careful not to confuse the term "normalization" with the term "normal vector." A normalized vector is any vector of unit length. A normal vector is any vector that is perpendicular to a surface, whether or not it is of unit length.

5.2.4.7 Dot Product and Projection

Vectors can be multiplied, but unlike scalars there are a number of different kinds of vector multiplication. In game programming, we most often work with the following two kinds of multiplication:

- the *dot product* (a.k.a. scalar product or inner product), and
- the *cross product* (a.k.a. vector product or outer product).

The dot product of two vectors yields a scalar; it is defined by adding the products of the individual components of the two vectors:

$$\mathbf{a} \cdot \mathbf{b} = a_x b_x + a_y b_y + a_z b_z = d \quad \text{(a scalar)}.$$

The dot product can also be written as the product of the magnitudes of the two vectors and the cosine of the angle between them:

$$\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}|\,|\mathbf{b}| \cos\theta.$$

The dot product is *commutative* (i.e., the order of the two vectors can be reversed) and *distributive* over addition:

$$\mathbf{a} \cdot \mathbf{b} = \mathbf{b} \cdot \mathbf{a};$$
$$\mathbf{a} \cdot (\mathbf{b} + \mathbf{c}) = \mathbf{a} \cdot \mathbf{b} + \mathbf{a} \cdot \mathbf{c}.$$

And the dot product combines with scalar multiplication as follows:

$$s\mathbf{a} \cdot \mathbf{b} = \mathbf{a} \cdot s\mathbf{b} = s(\mathbf{a} \cdot \mathbf{b}).$$

Vector Projection

If $\mathbf{u}$ is a unit vector ($|\mathbf{u}| = 1$), then the dot product ($\mathbf{a} \cdot \mathbf{u}$) represents the length of the *projection* of vector $\mathbf{a}$ onto the infinite line defined by the direction of $\mathbf{u}$, as shown in Figure 5.10. This projection concept works equally well in 2D or 3D and is highly useful for solving a wide variety of three-dimensional problems.

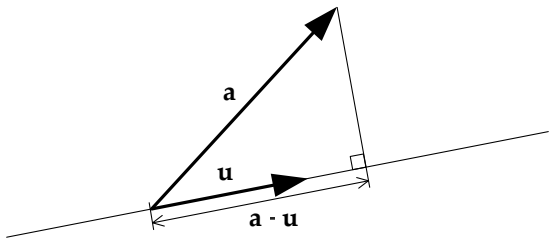

Figure 5.10. Vector projection using the dot product.

Magnitude as a Dot Product

The squared magnitude of a vector can be found by taking the dot product of

that vector with itself. Its magnitude is then easily found by taking the square root:

$$|\mathbf{a}|^2 = \mathbf{a} \cdot \mathbf{a};$$
$$|\mathbf{a}| = \sqrt{\mathbf{a} \cdot \mathbf{a}}.$$

This works because the cosine of zero degrees is 1, so $|\mathbf{a}|\,|\mathbf{a}|\cos\theta = |\mathbf{a}|\,|\mathbf{a}| = |\mathbf{a}|^2$.

Dot Product Tests

Dot products are great for testing if two vectors are collinear or perpendicular, or whether they point in roughly the same or roughly opposite directions. For any two arbitrary vectors **a** and **b**, game programmers often use the following tests, as shown in Figure 5.11:

- *Collinear*. $(\mathbf{a} \cdot \mathbf{b}) = |\mathbf{a}|\,|\mathbf{b}| = ab$ (i.e., the angle between them is exactly 0 degrees—this dot product equals $+1$ when **a** and **b** are *unit vectors*).
- *Collinear but opposite*. $(\mathbf{a} \cdot \mathbf{b}) = -ab$ (i.e., the angle between them is 180 degrees—this dot product equals -1 when **a** and **b** are unit vectors).
- *Perpendicular*. $(\mathbf{a} \cdot \mathbf{b}) = 0$ (i.e., the angle between them is 90 degrees).
- *Same direction*. $(\mathbf{a} \cdot \mathbf{b}) > 0$ (i.e., the angle between them is less than 90 degrees).
- *Opposite directions*. $(\mathbf{a} \cdot \mathbf{b}) < 0$ (i.e., the angle between them is greater than 90 degrees).

Some Other Applications of the Dot Product

Dot products can be used for all sorts of things in game programming. For example, let's say we want to find out whether an enemy is in front of the player character or behind him. We can find a vector from the player's position **P** to the enemy's position **E** by simple vector subtraction ($\mathbf{v} = \mathbf{E} - \mathbf{P}$). Let's assume we have a vector **f** pointing in the direction that the player is *facing*. (As we'll see in Section 5.3.10.3, the vector **f** can be extracted directly from the player's model-to-world matrix.) The dot product $d = \mathbf{v} \cdot \mathbf{f}$ can be used to test whether the enemy is in front of or behind the player—it will be positive when the enemy is in front and negative when the enemy is behind.

The dot product can also be used to find the height of a point above or below a plane (which might be useful when writing a moon-landing game for example). We can define a plane with two vector quantities: a point **Q** lying anywhere on the plane, and a unit vector **n** that is perpendicular (i.e., normal)

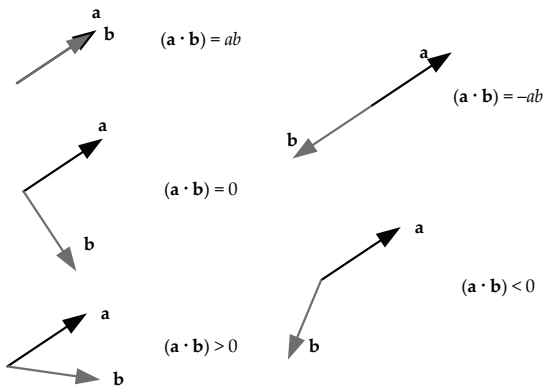

Figure 5.11. Some common dot product tests.

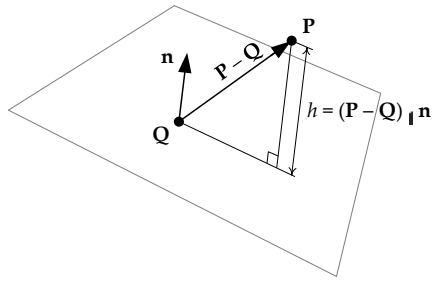

Figure 5.12. The dot product can be used to find the height of a point above or below a plane.

to the plane. To find the height h of a point $\mathbf{P}$ above the plane, we first calculate a vector from any point on the plane ($\mathbf{Q}$ will do nicely) to the point in question $\mathbf{P}$. So we have $\mathbf{v} = \mathbf{P} - \mathbf{Q}$. The dot product of vector $\mathbf{v}$ with the unit-length normal vector $\mathbf{n}$ is just the projection of $\mathbf{v}$ onto the line defined by $\mathbf{n}$. But that is exactly the height we're looking for. Therefore,

$$h = \mathbf{v} \cdot \mathbf{n} = (\mathbf{P} - \mathbf{Q}) \cdot \mathbf{n}. \tag{5.2}$$

This is illustrated in Figure 5.12.

5.2.4.8 Cross Product

The *cross product* (also known as the *outer product* or *vector product*) of two vectors yields another *vector* that is *perpendicular* to the two vectors being multiplied, as shown in Figure 5.13. The cross product operation is only defined in

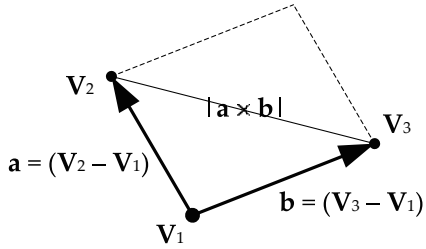

Figure 5.14. Area of a parallelogram expressed as the magnitude of a cross product.

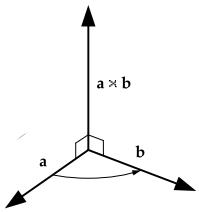

Figure 5.13. The cross product of vectors **a** and **b** (right-handed).

three dimensions:

$$\begin{aligned}\mathbf{a} \times \mathbf{b} &= \big[(a_y b_z - a_z b_y), (a_z b_x - a_x b_z), (a_x b_y - a_y b_x)\big] \\ &= (a_y b_z - a_z b_y)\mathbf{i} + (a_z b_x - a_x b_z)\mathbf{j} + (a_x b_y - a_y b_x)\mathbf{k}.\end{aligned}$$

Magnitude of the Cross Product

The magnitude of the cross product vector is the product of the magnitudes of the two vectors and the sine of the angle between them. (This is similar to the definition of the dot product, but it replaces the cosine with the sine.)

$$|\mathbf{a} \times \mathbf{b}| = |\mathbf{a}|\,|\mathbf{b}| \sin\theta.$$

The magnitude of the cross product $|\mathbf{a} \times \mathbf{b}|$ is equal to the area of the parallelogram whose sides are **a** and **b**, as shown in Figure 5.14. Since a triangle is one half of a parallelogram, the area of a triangle whose vertices are specified by the position vectors $\mathbf{V}_1$, $\mathbf{V}_2$ and $\mathbf{V}_3$ can be calculated as one half of the magnitude of the cross product of any two of its sides:

$$A_{\text{triangle}} = \tfrac{1}{2}\,\big|(\mathbf{V}_2 - \mathbf{V}_1) \times (\mathbf{V}_3 - \mathbf{V}_1)\big|\,.$$

Direction of the Cross Product

When using a right-handed coordinate system, you can use the *right-hand rule* to determine the direction of the cross product. Simply cup your fingers such that they point in the direction you'd rotate vector **a** to move it on top of vector **b**, and the cross product $(\mathbf{a} \times \mathbf{b})$ will be in the direction of your thumb.

Note that the cross product is defined by the *left-hand rule* when using a left-handed coordinate system. This means that the direction of the cross product changes depending on the choice of coordinate system. This might seem odd

at first, but remember that the handedness of a coordinate system does *not* affect the mathematical calculations we carry out—it only changes our *visualization* of what the numbers look like in 3D space. When converting from a right-handed system to a left-handed system or vice versa, the numerical representations of all the points and vectors stay the same, but one axis flips. Our visualization of everything is therefore mirrored along that flipped axis. So if a cross product just happens to align with the axis we're flipping (e.g., the z-axis), it needs to flip when the axis flips. If it didn't, the mathematical definition of the cross product itself would have to be changed so that the z-coordinate of the cross product comes out negative in the new coordinate system. I wouldn't lose too much sleep over all of this. Just remember: when *visualizing* a cross product, use the right-hand rule in a right-handed coordinate system and the left-hand rule in a left-handed coordinate system.

Properties of the Cross Product

The cross product is *not commutative* (i.e., order matters):

$$\mathbf{a} \times \mathbf{b} \neq \mathbf{b} \times \mathbf{a}.$$

However, it is *anti-commutative:*

$$\mathbf{a} \times \mathbf{b} = -(\mathbf{b} \times \mathbf{a}).$$

The cross product is distributive over addition:

$$\mathbf{a} \times (\mathbf{b} + \mathbf{c}) = (\mathbf{a} \times \mathbf{b}) + (\mathbf{a} \times \mathbf{c}).$$

And it combines with scalar multiplication as follows:

$$(s\mathbf{a}) \times \mathbf{b} = \mathbf{a} \times (s\mathbf{b}) = s(\mathbf{a} \times \mathbf{b}).$$

The Cartesian basis vectors are related by cross products as follows:

$$\begin{aligned} \mathbf{i} \times \mathbf{j} &= -(\mathbf{j} \times \mathbf{i}) = \mathbf{k} \\ \mathbf{j} \times \mathbf{k} &= -(\mathbf{k} \times \mathbf{j}) = \mathbf{i} \\ \mathbf{k} \times \mathbf{i} &= -(\mathbf{i} \times \mathbf{k}) = \mathbf{j} \end{aligned}$$

These three cross products define the direction of *positive rotations* about the Cartesian axes. The positive rotations go from x to y (about z), from y to z (about x) and from z to x (about y). Notice how the rotation about the y-axis "reversed" alphabetically, in that it goes from z to x (not from x to z). As we'll

see below, this gives us a hint as to why the matrix for rotation about the y-axis looks *inverted* when compared to the matrices for rotation about the x- and z-axes.

The Cross Product in Action

The cross product has a number of applications in games. One of its most common uses is for finding a vector that is perpendicular to two other vectors. As we'll see in Section 5.3.10.2, if we know an object's local unit basis vectors, ($\mathbf{i}_{\text{local}}$, $\mathbf{j}_{\text{local}}$ and $\mathbf{k}_{\text{local}}$), we can easily find a matrix representing the object's orientation. Let's assume that all we know is the object's $\mathbf{k}_{\text{local}}$ vector—i.e., the direction in which the object is facing. If we assume that the object has no roll about $\mathbf{k}_{\text{local}}$, then we can find $\mathbf{i}_{\text{local}}$ by taking the cross product between $\mathbf{k}_{\text{local}}$ (which we already know) and the world-space up vector $\mathbf{j}_{\text{world}}$ (which equals $[\,0\;1\;0\,]$). We do so as follows: $\mathbf{i}_{\text{local}} = \text{normalize}(\mathbf{j}_{\text{world}} \times \mathbf{k}_{\text{local}})$. We can then find $\mathbf{j}_{\text{local}}$ by simply crossing $\mathbf{i}_{\text{local}}$ and $\mathbf{k}_{\text{local}}$ as follows: $\mathbf{j}_{\text{local}} = \mathbf{k}_{\text{local}} \times \mathbf{i}_{\text{local}}$.

A very similar technique can be used to find a unit vector normal to the surface of a triangle or some other plane. Given three points on the plane, $\mathbf{P}_1$, $\mathbf{P}_2$ and $\mathbf{P}_3$, the normal vector is just $\mathbf{n} = \text{normalize}\left((\mathbf{P}_2 - \mathbf{P}_1) \times (\mathbf{P}_3 - \mathbf{P}_1)\right)$.

Cross products are also used in physics simulations. When a force is applied to an object, it will give rise to rotational motion if and only if it is applied off-center. This rotational force is known as a *torque*, and it is calculated as follows. Given a force $\mathbf{F}$, and a vector $\mathbf{r}$ from the center of mass to the point at which the force is applied, the torque $\mathbf{N} = \mathbf{r} \times \mathbf{F}$.

5.2.4.9 Pseudovectors and Exterior Algebra

We mentioned in Section 5.2.2 that the cross product doesn't actually produce a vector—it produces a special kind of mathematical object known as a *pseudovector*. The difference between a vector and a pseudovector is pretty subtle. In fact, you can't tell the difference between them at all when performing the kinds of transformations we normally encounter in game programming—translation, rotation and scaling. It's only when you *reflect* the coordinate system (as happens when you move from a left-handed coordinate system to a right-handed system) that the special nature of pseudovectors becomes apparent. Under reflection, a vector transforms into its mirror image, as you'd probably expect. But when a pseudovector is reflected, it transforms into its mirror image and also *changes direction*.

Positions and all of the derivatives thereof (linear velocity, acceleration, jerk) are represented by true vectors (also known as *polar vectors* or *contravariant vectors*). Angular velocities and magnetic fields are represented by pseu-

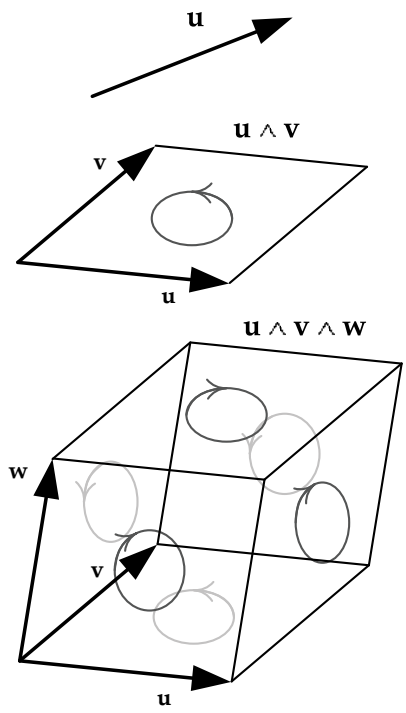

Figure 5.15. In the exterior algebra (Grassman algebra), a single wedge product yields a pseudovector or bivector, and two wedge products yield a pseudoscalar or trivector.

dovectors (also known as *axial vectors*, *covariant vectors*, *bivectors* or *2-blades*). The surface normal of a triangle (which is calculated using a cross product) is also a pseudovector.

It's pretty interesting to note that the cross product ($\mathbf{A} \times \mathbf{B}$), the scalar triple product ($\mathbf{A} \cdot (\mathbf{B} \times \mathbf{C})$) and the determinant of a matrix are all inter-related, and pseudovectors lie at the heart of it all. Mathematicians have come up with a set of algebraic rules, called an *exterior algebra* or *Grassman algebra*, which describe how vectors and pseudovectors work and allow us to calculate areas of parallelograms (in 2D), volumes of parallelepipeds (in 3D), and so on in higher dimensions.

We won't get into all the details here, but the basic idea of Grassman algebra is to introduce a special kind of vector product known as the *wedge product*, denoted $\mathbf{A} \wedge \mathbf{B}$. A pairwise wedge product yields a pseudovector and is equivalent to a cross product, which also represents the *signed area* of the parallelogram formed by the two vectors (where the sign tells us whether we're rotating from $\mathbf{A}$ to $\mathbf{B}$ or vice versa). Doing two wedge products in a row, $\mathbf{A} \wedge \mathbf{B} \wedge \mathbf{C}$,

is equivalent to the scalar triple product $\mathbf{A} \cdot (\mathbf{B} \times \mathbf{C})$ and produces another strange mathematical object known as a *pseudoscalar* (also known as a *trivector* or a *3-blade*), which can be interpreted as the *signed volume* of the parallelepiped formed by the three vectors (see Figure 5.15). This extends into higher dimensions as well.

What does all this mean for us as game programmers? Not too much. All we really need to keep in mind is that some vectors in our code are actually pseudovectors, so that we can transform them properly when changing handedness, for example. Of course if you really want to geek out, you can impress your friends by talking about *exterior algebras* and *wedge products* and explaining how cross products aren't really vectors. Which might make you look cool at your next social engagement …or not.

For more information, see http://en.wikipedia.org/wiki/Pseudovector, http://en.wikipedia.org/wiki/Exterior_algebra, and http://www.terathon.com/gdc12_lengyel.pdf.

5.2.5 Linear Interpolation of Points and Vectors

In games, we often need to find a vector that is midway between two known vectors. For example, if we want to smoothly animate an object from point **A** to point **B** over the course of two seconds at 30 frames per second, we would need to find 60 intermediate positions between **A** and **B**.

A *linear interpolation* is a simple mathematical operation that finds an intermediate point between two known points. The name of this operation is often shortened to LERP. The operation is defined as follows, where β ranges from 0 to 1 inclusive:

$$\begin{aligned}\mathbf{L} &= \text{LERP}(\mathbf{A}, \mathbf{B}, \beta) = (1-\beta)\mathbf{A} + \beta\mathbf{B} \\ &= \left[(1-\beta)A_x + \beta B_x, \quad (1-\beta)A_y + \beta B_y, \quad (1-\beta)A_z + \beta B_z\right]\end{aligned}$$

Geometrically, $\mathbf{L} = \text{LERP}(\mathbf{A}, \mathbf{B}, \beta)$ is the position vector of a point that lies β percent of the way along the line segment from point **A** to point **B**, as shown in Figure 5.16. Mathematically, the LERP function is just a *weighted average* of the two input vectors, with weights $(1-\beta)$ and β, respectively. Notice that the weights always add to 1, which is a general requirement for any weighted average.

5.3 Matrices

A *matrix* is a rectangular array of $m \times n$ scalars. Matrices are a convenient way of representing linear transformations such as translation, rotation and scale.